Appello del 25 Gennaio 2019



Politecnico di Milano
Anno accademico 2018-2019

Ingegneria del Software

Cognome:		LAUREANDO ☐
Nome:	Matricola:	
Sezione (segnarne una): ☐ Cugola	☐ Margara	☐ San Pietro

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare solo i fogli distribuiti utilizzando il retro delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. non verranno in nessun caso presi in considerazione. È possibile scrivere in matita.
3. È possibile consultare liberamente libri, manuali o appunti. È proibito l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, smartwatch, ...).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. La consegna di un compito gravemente insufficiente comporta l'impossibilità a partecipare all'appello successivo.
6. Tempo a disposizione: 2h.

Esercizio 1:

Esercizio 2:

Esercizio 3:

Esercizio 4:

Esercizio 1

Si consideri il seguente tipo di dato astratto che definisce una `MultiList<T>`. Una `MultiList<T>` è un atomo (ovvero un elemento di un generico tipo `T`) oppure una lista di `MultiList<T>`.
Per esempio, `(1 (2) 3 4 ((5 6) 7))` è una rappresentazione testuale di una `MultiList<Integer>`, il cui primo elemento (*first*) è l'atomo `1` e il cui resto (*rest*) è la `MultiList ((2) 3 4 ((5 6) 7))`, ovvero la `MultiList` privata del suo primo elemento.
Altri esempi di `MultiList<Integer>` sono l'atomo `3` e la `MultiList (3)`. Si noti che queste due `MultiList<Integer>` sono diverse: la prima è una `MultiList` "atomica", mentre la seconda è una "vera" `MultiList` che contiene al suo interno l'atomo `3`.

```
public class MultiList<T> {
  // Crea una MultiList vuota
  public MultiList();

  // Crea una MultiList composta dal solo atomo x (non nullo)
  public MultiList(T x);

  // Ritorna true se this e' vuota
  public /*@ pure @*/ boolean isEmpty()

  // Ritorna true se this e' un atomo
  public /*@ pure @*/ boolean isAtom()

  // Ritorna il valore dell'atomo se this e' un atomo
  // Lancia un'eccezione opportuna in ogni altro caso
  public /*@ pure @*/ T getAtom() throws EmptyException, MalformedListException ;

  // Ritorna il primo elemento di this, se this non e' un atomo e non e' vuota.
  // Lancia un'eccezione opportuna in ogni altro caso
  public /*@ pure @*/ MultiList<T> first() throws EmptyException, MalformedListException ;

  // Ritorna la MultiList<T> che contiene tutti gli elementi
  // di this tranne il primo.
  // Lancia un'eccezione opportuna se this e' vuota o se e' un atomo
  public /*@ pure @*/ MultiList<T> rest() throws EmptyException, MalformedListException ;

  // Aggiunge l'elemento x (una MultiList<T> non nulla) come primo elemento di this.
  // Lancia una MalformedListException se this e' un atomo
  public void addFirst(MultiList<T> x) throws MalformedListException ;

  // Rimuove il primo elemento (una MultiList<T>) da this.
  // Lancia un'eccezione opportuna se this e' vuota o se e' un atomo
  public void removeFirst() throws EmptyException, MalformedListException ;

  // Ritorna true se this contiene la MultiList<T> x come elemento
  // a qualsiasi livello di annidamento.
  // Lancia una MalformedListException se this e' un atomo
  // (mentre ritorna false se this e' empty)
  public /*@ pure @*/ boolean contains(MultiList<T> x) MalformedListException ;

  // Ritorna true se this e that sono entrambi atomi e hanno
  // lo stesso valore (this.atom.equals(that.atom))
  // oppure se sono entrambe MultiList "vere" che contengono
  // gli stessi elementi nello stesso ordine
  @Override
  public /*@ pure @*/ boolean equals(Object that) ;

  // Ritorna una copia di this, tale per cui this.equals(copy())
  public /*@ pure @*/ MultiList<T> copy() ;
}
```

Domanda a)

Si specifichino in JML i metodi `addFirst`, `removeFirst` e `contains`.

Soluzione

```
//@requires x!=null;
//@ensures !isAtom() && first().equals(x) && rest().equals(\old(copy()));
//@signals(MalformedListException e) \old(isAtom()) && copy().equals(\old(copy()))
public void addFirst(MultiList<T> x) throws MalformedListException;

//@ensures !isAtom() && !isEmpty() && this.equals(\old(rest()));
//@signals(EmptyException e) \old(isEmpty()) && isEmpty();
//@signals(MalformedListException e) \old(isAtom()) && copy().equals(\old(copy()))
public void removeFirst() throws EmptyException, MalformedListException ;

//@requires x!=null;
//@ensures !this.isAtom() && (\result==true <==> (!this.isEmpty() &&
//@        (first().equals(x) ||
//@         (!first().isAtom() && first().contains(x)) ||
//@         (rest()!=null && rest().contains(x)))))
//@signals(MalformedListException e) \old(isAtom()) && copy().equals(\old(copy()))
public /*@ pure @*/ boolean contains(MultiList<T> x) MalformedListException {
```

Domanda b)

Si consideri la seguente implementazione di una `MultiList<T>`.

```
public class MultiList<T> {
  private T atom;
  private List<MultiList<T>> list;

  public MultiList() {
    atom = null; list = new ArrayList<MultiList<T>>();
  }
  public MultiList(T x) { // richiede che x non sia null
    atom = x; list = null;
  }
  public /*@ pure @*/ boolean isEmpty() {
    return atom==null && list.isEmpty();
  }
  public /*@ pure @*/ boolean isAtom() {
    return atom != null;
  }
  public /*@ pure @*/ T getAtom() throws EmptyException, MalformedListException {
    if(isEmpty()) throw new EmptyException();
    if(!isAtom()) throw new MalformedListException();
    return atom;
  }
  public /*@ pure @*/ MultiList<T> first() throws EmptyException, MalformedListException {
    if(isEmpty()) throw new EmptyException();
    if(isAtom()) throw new MalformedListException();
    return list.get(0);
  }
  public /*@ pure @*/ MultiList<T> rest() throws EmptyException, MalformedListException {
    if(isEmpty()) throw new EmptyException();
    if(isAtom()) throw new MalformedListException();
    if(list.size()==1) return null;
    MultiList<T> res = new MultiList<T>();
    res.list.addAll(list.sublist(1,list.size()));
    return res;
  }
```

```
  public void addFirst(MultiList<T> x) throws MalformedListException {
    if(isAtom()) throw new MalformedListException();
    list.add(0, x);
  }
  public void removeFirst() throws EmptyException, MalformedListException {
    if(isEmpty()) throw new EmptyException();
    if(isAtom()) throw new MalformedListException();
    list.remove(0);
  }
  public /*@ pure @*/ boolean contains(MultiList<T> x) MalformedListException {
    if(isAtom()) throw new MalformedListException();
    if(isEmpty()) return false;
    if(first().equals(x)) return true;
    if(!first().isAtom() && first().contains(x)) return true;
    return !rest().isEmpty() && rest().contains(x);
  }
  @Override
  public /*@ pure @*/ boolean equals(Object that) {
    // .....
  }
  public /*@ pure @*/ MultiList<T> copy() {
    // .....
  }
}
```

Scrivere in JML l'invariante di rappresentazione (Rep Invariant) per `MultiList<T>`.

Soluzione

```
//@private invariant (atom==null <==> list!=null)  &&
//@                  (list!=null ==>
//@                   (\forall int i; 0<=i && i<list.size();
//@                      list.get(i)!=null));
```

Servirebbe anche una condizione di non circolarita' della MultiList. Questa si puo' ottenere ad esempio definendo un nuovo predicato `boolean /*@ pure @*/ containsEqual(MultiList<T> lista, MultiList<T> elem)`, identico a `contains()` ma usando equals:

```
//@ensures !this.isAtom() && (\result==true <==> (!this.isEmpty() &&
//@        (first() == x) ||
//@         (!first().isAtom() && first().containsEqual(x)) ||
//@         (!rest().isEmpty() && rest().containsEqual(x)))))
//@boolean /*@ pure @*/ containsEqual(MultiList<T> lista, MultiList<T> elem)
```

e aggiungendo inoltre un'opportuna condizione nel RI:

```
(\forall int i; 0<=i && i<list.size();
    !(list.get(i).containsEqual(list.get(i))))
```

Tuttavia, l'implementazione di `addFirst` puo' inserire un oggetto anche se questo gia' esistente nella lista, pertanto violando questa parte dell'invariante.

Domanda c)

Si consideri la classe `NullMultiList<T>` che modifica `MultiList<T>` rimuovendo il vincolo che una `MultiList<T>` non possa contenere atomi nulli. La classe `NullMultiList<T>` è un'estensione valida di `MultiList<T>` secondo il principio di sostituzione di Liskov? Perché? È vero il contrario?

Soluzione

`NullMultiList<T>` indebolisce la pre-condizione dei metodi in `MultiList<T>` (in particolare, il costruttore di un atomo e il metodo `addFirst`), ma indebolisce anche la post-condizione del metodo `first`, dato che questo potrebbe ritornare un valore nullo. Di conseguenza `NullMultiList<T>` non è un'estensione valida di `MultiList<T>` e nemmeno il contrario.

Esercizio 2

Si consideri la seguente interfaccia `TenIntArray` che rappresenta un array di interi che può contenere al massimo 10 elementi interi. Un chiamante può accedere agli elementi dell'array mediante il loro indice (metodo `get()`), aggiungere un elemento in ultima posizione (metodo `add()`) e rimuovere l'elemento in ultima posizione (metodo `remove()`). Se il chiamante prova ad aggiungere un elemento in un `TenIntArray` pieno, viene sospeso fino a quando un altro thread rimuove un elemento.

```
public interface TenIntArray {

  /**
   * Ritorna il valore contenuto in posizione pos, se presente,
   * altrimenti lancia una InvalidIndexException.
   */
  public int get(int pos) throws InvalidIndexException;

  /**
   * Rimuove l'ultimo elemento dall'array.
   * Se l'array e' vuoto non fa nulla.
   */
  public void remove();

  /**
   * Aggiunge val nella prima posizione libera dell'array.
   * Se l'array e' pieno, blocca il chiamante fino a quando una
   * posizione si libera.
   */
  public void add(int val);
}
```

Domanda a)

Si completi il codice della classe `TenIntArrayImpl` che implementa `TenIntArray`. Si usino solo i meccanismi base della sincronizzazione di Java senza sfruttare classi e pacchetti di `java.util.concurrent.XXX`.

```
public class TenIntArrayImpl implements TenIntArray {
  private final int array[];
  private int size; // Indica l'indice della prima posizione libera nell'array

  public TenIntArrayImpl() {
    array = new int[10];
    size = 0;
  }

  ...
}
```

Soluzione

```
public class TenIntArrayImpl implements TenIntArray {
  private final int array[];
  private int size; // Indica l'indice della prima posizione libera nell'array
```

```
  public TenIntArrayImpl() {
    array = new int[10];
    size = 0;
  }

  @Override
  public synchronized int get(int pos) throws InvalidIndexException {
    if (pos < 0 || pos >= size) {
      throw new InvalidIndexException();
    } else {
      return array[pos];
    }
  }

  @Override
  public synchronized void remove() {
    if (size > 0) {
      size--;
      notifyAll();
    }
  }

  @Override
  public synchronized void add(int val) {
    while (size == 10) {
      try {
        wait();
      } catch (final InterruptedException e) {
        e.printStackTrace();
      }
    }
    array[size] = val;
    size++;
  }

}
```

Domanda b)

Si consideri la seguente classe `Test` che utilizza `TenIntArray`. Indicare per quali valori di `k` si può garantire che tutti i thread lanciati nel `main()` terminino.

```
public class Test implements Runnable {
  private final TenIntArray t;
  private final int id;

  public Test(TenIntArray t, int id) {
    this.t = t;
    this.id = id;
  }

  @Override
  public void run() {
    if (id%2!=0) {
      t.add(id);
    } else {
      t.remove();
    }
  }

  public static void main(String[] args) {
```

```
    final int k = ???;
    final TenIntArray arr = new TenIntArrayImpl();
    for (int i = 1; i <= k; i++) {
      final Thread thread = new Thread(new Test(arr, i));
      thread.start();
    }
  }
}
```

Soluzione

Alcuni thread potrebbero non terminare per `k>20`. In questo caso, potrebbero essere eseguiti per primi tutti i thread con indice `id` pari, che lasciano `t` vuoto. Successivamente 10 thread con `id` dispari potrebbero riempire `t`, bloccando ogni successivo thread con `id` dispari che invoca `t.add(id)`.

Esercizio 3

Si considerino le seguenti dichiarazioni di tipi Java.

```
interface Product {
  public double getPrice();
}

class Drink implements Product {
  private double price;
  public Drink(double price) { this.price = price; }
  public double getPrice() { return price; }
  public boolean moreExpensive(Drink d) { return this.getPrice() > d.getPrice(); }
}

class Coca extends Drink {
  public Coca(double price) { super(price); }
  public int getCalories() { return 200; }
}

class CocaZero extends Coca {
  public CocaZero(double price) { super(price); }
  public int getCalories() { return 0; }
  public boolean moreExpensive(CocaZero c) { return false; }
}
```

Si consideri poi il seguente codice Java che utilizza le classi sopra.

```
public class Main {
   public static void main(String[] args) {
1.   Product p;
2.   Drink d1, d2;
3.   Coca c1, c2;

4.   p = new Coca(2.0);
5.   d1 = p;
6.   d2 = new Drink(1.5);
7.   c1 = new Coca(2.1);
8.   c2 = new CocaZero(2.5);

9.   System.out.println(p.getPrice());
10.  System.out.println(d1.getPrice());
11.  System.out.println(d2.getPrice());
12.  System.out.println(c1.getPrice());
13.  System.out.println(c2.getPrice());
```

```
14.  System.out.println(p.getCalories());
15.  System.out.println(c1.getCalories());
16.  System.out.println(c2.getCalories());

17.  System.out.println(c1.moreExpensive(d2));
18.  System.out.println(c1.moreExpensive(c2));

19.  CocaZero cz = new CocaZero(2.0);
20.  System.out.println(c2.moreExpensive(cz));
   }
}
```

Domanda a)

Si indichino quali righe del metodo `main` generano un errore di compilazione e perchè.

Soluzione

Linea 5. non è possibile assegnare ad un `Drink` un `Product` (`Product` è sopra-classe, non sotto-classe di `Drink`).

Linea 10. eliminata la linea 5 anche la 10 deve essere eliminata.

Linea 14. La classe `Product` non ha il metodo `getCalories`.

Domanda b)

Supponendo che tutte le righe che provocano errori in fase di compilazione siano state rimosse, illustrare l'output del programma, spiegando quali metodi vengono invocati e perchè.

Solution

Output:

```
2.0
1.5
2.1
2.5
200
0
true
false
true
```

Si noti in particolare l'ultimo risultato: viene invocato il metodo `moreExpensive` di `Drink` e non quello di `CocaZero` visto che `CocaZero` non ridefinisce il metodo `moreExpensive` di `Drink` ma ne aggiunge un altro con diversa signature (non si applica quindi il binding dinamico).

Esercizio 4

Si consideri la seguente funzione:

```
public static int fun(int a, int b) {
  int temp = 0;
  if (a > b) {
    temp++;
  }
  if (a + b > 10) {
    temp += a;
  } else {
    temp += b;
```

```
  }
  return temp;
}
```

Si definisca un insieme di casi di test per ciascuna delle seguenti richieste:

a) L'insieme copre tutte le istruzioni (statement coverage);

b) L'insieme copre tutti i branch/decisioni (branch/edge coverage);

c) L'insieme copre tutti i cammini (path coverage).

Soluzione

a) T1(a=10, b=5) T2(a=1, b=1)

b) T1(a=10, b=5) T2(a=1, b=1)

c) T1(a=10, b=5) T2(a=1, b=1) T3(a=2, b=1) T4(a=10, b=20)

Il caso T1 copre le istruzioni dentro i due rami `if`. Il caso T2 copre le istruzioni nel ramo `else` (senza entrare in nessuno dei due rami `if`). T1 e T2 sono quindi sufficienti sia per coprire tutte le istruzioni che tutte le decisioni.
Per coprire tutti i cammini occorre anche considerare il caso in cui venga eseguito il primo ramo `if` e il secondo ramo `else` (cammino coperto da T3) e il caso in cui non venga eseguito il primo ramo `if` e venga invece eseguito il secondo (cammino coperto da T4).